CORE-SG: Efficient Computation of Multiple MSTs for Density-Based Methods

Antonio Cavalcante Araujo Neto
Department of Computing Science
University of Alberta
antonio.cavalcante@ualberta.ca

Murilo Coelho Naldi
Department of Computing Science
Federal University of São Carlos
naldi@ufscar.br

Ricardo J. G. B. Campello
School of Math. and Phys. Science
University of Newcastle
ricardo.campello@newcastle.edu.au

Jörg Sander
Department of Computing Science
University of Alberta
jsander@ualberta.ca

2022 IEEE 38th International Conference on Data Engineering (ICDE) | 978-1-6654-0883-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICDE53745.2022.00076

***Abstract*—Several popular density-based methods for unsupervised and semi-supervised learning tasks, including clustering and classification, can be formulated as instances of a framework that is based on the processing of a minimum spanning tree of the data, where the edge weights correspond to a form of (unnormalized) density estimate *w.r.t.* a smoothing parameter m_{pts}. While density-based methods are considered to be robust *w.r.t.* m_{pts} in the sense that small changes in its value usually lead to slight or no changes in the resulting structure, wider ranges of m_{pts} values may lead to different results that a user would like to analyze before choosing the most suitable value for a given data set or application. However, to explore multiple results for a range of m_{pts} values, until recently, one had to re-run the density-based method for each value in the range independently, which is computationally inefficient. This paper proposes a new computationally efficient approach to compute multiple density-based minimum spanning trees *w.r.t.* a set of m_{pts} values by leveraging a graph obtained from a single run of the density-based algorithm, without the need for re-runs of the original algorithm. We present theoretical and experimental results that show that our approach overcomes the drawbacks of the previous state-of-the-art, and it is considerably superior in runtime and graph size while being easier to implement. Our experimental evaluation using synthetic and real data shows that our strategy can lead to speed-up factors of hundreds to thousands of times on the computation of density-based minimum spanning trees.**

***Keywords*-density-based methods, k-nearest neighbors, minimum spanning tree, clustering, semi-supervised clustering**

I. Introduction

Density-based methods for unsupervised and semi-supervised learning are popular options for exploratory data analysis in many application scenarios [1]–[6]. They can capture contiguous regions in a data space where the point density does not drop below a certain threshold, thus being able, e.g., to decompose a data set into groups of data points that help understand the underlying structure of the data while requiring little to no prior knowledge [7]. In the unsupervised setting, density-based clustering methods such as DBSCAN [8], OPTICS [9], and HDBSCAN* [10], [11] are well-known for their ability to find density-based clusters and discriminate clusters from noise, while being statistically sound. These algorithms have been successfully applied in a variety of fields, like robotics [12], genetics [13], human behaviour [14], chemistry [15], astronomy, and astrophysics [16], among many others. When a small portion of labeled data objects (i.e., ground-truth information) is available, semi-supervised clustering and semi-supervised classification methods can effectively make use of this information and take it into account when decomposing the data set into groups (clusters or classes) while also considering the underlying density of the data [17].

Motivation. Gertrudes *et al.* [18] have shown that a variety of unsupervised and semi-supervised clustering algorithms (including HDBSCAN*), as well as semi-supervised classification algorithms, can be described from a density-based perspective as instances or derivations of a unified framework that has as its common core the (direct or indirect) processing of a *Minimum Spanning Tree* (MST) of the data in a transformed data space. The edge weights of the MST in this space correspond to estimates of the density connecting the corresponding pairs of data objects. The MST is typically obtained by *dynamically* computing edge weights for each pair of data points, representing adjacent vertices of a conceptual, complete graph of the data that does not need to be materialized. The edge weights depend on a single user-defined parameter, m_{pts}, which acts as a classic smoothing factor for non-parametric density estimates. While relatively robust, setting this parameter can be challenging in practice because: (a) in the unsupervised and semi-supervised learning scenarios, at most a small portion of labeled data (if any) is available, making the application of supervised model selection approaches such as cross-validation difficult or impossible; and (b) very different values of m_{pts} can lead to very different results, and a user typically does not know a suitable value in advance. Moreover, it is fairly common in data mining tasks to spend considerable time exploring parameter values. The authors in [6], [19] have presented a collection of examples and visualizations that illustrate cases where different ranges of m_{pts} values yield different results.

A possible approach to circumvent the choice of a particular, potentially unsuited value of m_{pts} consists of performing exploratory analysis of multiple density-based results. This

2375-026X/22/$31.00 ©2022 IEEE
DOI 10.1109/ICDE53745.2022.00076

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

approach requires building the "density-based MST" (which can then be subsequently processed by each algorithm that relies on this graph) multiple times for several values of m_{pts}, allowing for a thorough, posterior analysis of the results. To be effective, one would want/need to consider a set or a range of m_{pts} values to make sure that suitable values are included and also to allow for a more in-depth exploration of the data. However, the computational cost of this approach can be prohibitive, especially for large data sets, because it requires the computation of a number of $MSTs$ from a *complete graph*, one for each value of m_{pts}. Although this complete graph does not have to be explicitly materialized and stored in memory, its $O(n^2)$ edges (for n data objects) will eventually need to be dynamically processed. These edges are weighted according to the input parameter m_{pts}, and such a dependence requires the re-computation of the edge weights and, accordingly, the re-computation of the MST for each different value of m_{pts}. These re-computations may be sped up with the reuse of information from the previous run, at the cost of keeping the data about the m_{pts} nearest neighbors of each data point as well as the complete graph in memory. However, such memory usage requirements can quickly become prohibitive in practical applications with increasing data set sizes.

To reduce the computational requirements of reusing the complete graph when finding additional density-based $MSTs$, a smaller graph can be used instead. The authors in [19], [20] proposed a method to efficiently compute density-based $MSTs$ for a whole range of m_{pts} values following this approach. Specifically, instead of resorting to the complete graph with different edge weights for each value of m_{pts}, they proposed to pre-compute a *Relative Neighborhood Graph* (RNG). This graph does not contain edges from the complete graph that are guaranteed not to be part of any $MSTs$ for the given range of m_{pts} values. Thus, all the additional $MSTs$ can be extracted from the single RNG efficiently, since the RNG typically has far fewer edges than the complete graph.

Although the RNG-based approach showed to be useful in practical scenarios, it has shortcomings that may limit its efficiency or even preclude its application, including: (i) the RNG will eventually approach the complete graph in terms of the number of edges as the dimensionality of the data increases, which incurs an overhead of pre-computing and storing a large RNG for a diminishing gain in the overall performance of computing $MSTs$; (ii) the RNG is based on geometric properties and, in order to be efficient, an implementation must rely on assumptions that restrict its applicability, namely, the data must be real-valued feature-vectors and a metric must be used as distance measure. These assumptions prevent the application of the method in scenarios where the data cannot be described as a set of points in a system of coordinates, as well as in scenarios requiring distance measures that do not satisfy the triangle inequality.

Contributions. Here in this paper, we describe original findings that overcome these shortcomings, and we propose a new, much more general, and, at the same time, more efficient solution to computing multiple density-based $MSTs$ for different m_{pts} values. Surprisingly, this solution relies only on information that can be shown to be available when computing a single density-based MST, corresponding to the largest value of m_{pts} in a given range of candidate values. It avoids all the limitations and intricacies involved in building an RNG and, more importantly, it leads to a sub-graph of the RNG, typically with a smaller number of edges. This reduced number of edges translates to an even more efficient way of computing multiple results of unsupervised and semi-supervised density-based algorithms *w.r.t.* a set of m_{pts} values, leading to speed-up factors that range from hundreds to thousands.

In particular, we show that multiple density-based solutions for a variety of m_{pts} values can be exactly extracted from the union of two graphs: (1) the MST of the complete graph in the transformed space of density estimates where edge weights are calculated *w.r.t.* to the largest candidate value of m_{pts} (defined here as k_{max}) and (2) the *k-Nearest Neighbor Graph* (k-NNG), which connects each object in a data set with its k nearest neighbors in the original data space according to a given dissimilarity/distance measure. We also formally prove that the resulting graph, $CORE$-SG, is a sub-graph of the RNG. The major implications of these results are that:

1) there is a linear upper bound on the number of edges in the pre-computed graph *w.r.t.* the number of data objects, n (assuming $k_{max} << n$, as expected in practical applications). This is a major improvement over the quadratic upper bound of the complete graph or the RNG;
2) the graph can be obtained from information that is readily available after a single run of a density-based algorithm that relies on an MST *w.r.t.* m_{pts} (e.g., HDBSCAN*), simply by setting $m_{pts} = k_{max}$;
3) our approach does not make restrictive assumptions about the data representation or about the distance measure adopted, as opposed to the RNG-based approach;
4) one can leverage the properties of $CORE$-SG to run an optional optimization step and obtain the minimal replacement graph possible.

The rest of this paper is organized as follows. Section II discusses the related work. Section III describes our theoretical findings and proves their correctness. In Section IV we describe how these findings can be used to more efficiently tackle the problem of interest in this paper. Section V presents and discusses our experimental evaluation. Finally, Section VI concludes the paper and provides directions for future work.

II. Related Work

There has been a variety of work in the literature aimed at speeding/scaling up density-based learning algorithms, especially for large data sets [21]–[26]. Existing methods resort, e.g., to spatial information, data structures, sampling, and/or parallel and distributed techniques to efficiently compute a single density-based model. The contribution of our work has a different focus, namely: efficiently computing *additional* density-based models for a variety of m_{pts} values from an initial density-based model. The initial model can be obtained using any existing algorithm for computing a density-based

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

MST of the complete graph *w.r.t.* a single m_{pts} value. Thus, our method is not an alternative to density-based learning approaches in the literature, it rather complements them.

The work that is most related to ours is [6], [19], [20], where the authors replace the complete graph in HDBSCAN* with a typically much smaller graph, the RNG, in order to speed up the computation of multiple density-based clustering hierarchies for a range of values of the parameter m_{pts}. It was shown that the RNG-based strategy could be more than 60 times faster than a naive approach in which HDBSCAN* is run for each value of m_{pts} individually. Suitable visualizations for exploratory analysis were also devised in [6], [19], which can be readily used with our current work as well. In Section IV-D we discuss in more detail the novelty and contributions of our current proposal as compared to the RNG-based approach.

The need to efficiently produce unsupervised learning results across multiple density profiles recently motivated the work in [5], which proposed an index-based approach to speed up the execution of SCAN [27] (an adaptation of DBSCAN for *graphs*, i.e., *network clustering*) for different values of its density-control parameters, μ and ε. The proposed index can be reused to run SCAN for different values of μ and ε with significant computational savings. This approach is similar in intent to our work in that it proposes a data structure that allows to efficiently re-run algorithms for unsupervised learning (in this case a single algorithm) for multiple parameter values. However, the method is very specific to finding structurally-connected components in networks, and is not applicable to more general clustering tasks or other density-based tasks.

The authors in [28] proposed a clustering procedure based on the union of the Euclidean MST and a k-nearest neighbor graph, k-NNG, and studied the influence of the parameter k to find the smallest value of k, $\tilde{k}$, such that the k-NNG contains the MST, in order to identify clusters. They argue that the MST is included (as a sub-graph) in the k-NNG for $\tilde{k} = O(\log n)$ when no clustering structure exists in the data, yet this will only occur for much higher values of k if two or more well-separated clusters exist. Accordingly, for a data set with n objects, the removal of edges from the Euclidean MST that are not contained in the k-NNG, for $k = O(\log n)$, may lead to the identification of clusters in the data as the resulting connected components. Based on this assumption, a number of variations of an "MST-*k*NN" clustering algorithm were proposed in the literature (*e.g.*, [29], [30]), where the data is represented by a complete weighted graph with edge weights given by the similarities between vertices (data objects). These methods initially partition the data by using the above heuristic for removing certain edges from the Euclidean MST, and then try to refine the solution using different strategies to obtain a final "flat" partition of the data into clusters. While these approaches also use a form of MST and a k-NNG, they simply introduce heuristic procedures to remove certain edges from an Euclidean MST in order to obtain a flat partition of the data. Recalling that Euclidean $MSTs$ are equivalent to Single-Linkage (SL) clustering [31], these heuristics will result in flat partitions that suffer from the well-known shortcomings of SL solutions, most noticeably, the *chaining effect* and high sensitivity to noise [32]. In contrast, we propose a method that allows for the efficient computation of a range of $MSTs$ over a different graph, defined in a transformed space representing multiple density-based structures of the data (not required to be Euclidean), and such $MSTs$ can be turned into hierarchical or flat clustering solutions that do not suffer from the aforementioned SL shortcomings.

In [33], the authors combine density-based clustering concepts, as defined by DBSCAN, with spectral clustering to find high density clusters. The overall idea consists of minimizing a graph cut so that the average density of the obtained clusters is maximized, which results in clusters that are similar to connected components of *core-points* as defined by DBSCAN. In spectral clustering, the dimensionality of the data is reduced through the spectrum of the similarity matrix of a suitable graph, representing the data. One graph that plays this role is the k-NNG [34]. Veenstra *et al.* [35] show that the quality of the resulting partition can be negatively affected by certain values of k, since the k-NNG may not be a connected graph for these values. To guarantee connectivity, the authors propose to combine the k-NNG with the Euclidean MST. While these approaches also use a form of MST and a k-NNG to improve clustering, they are orthogonal to ours. By representing our graphs as similarity matrices, one could also combine our method with spectral clustering to explore multiple spectral clustering solutions for a variety of connected graphs representing the density-based structure of the data.

III. Theoretical Foundation

The unified framework of density-based methods for semi-supervised clustering and classification in [18] formalizes a relation between density-based clustering methods and the graph-based approach for transductive classification, while showing that a variety of popular algorithms can all be interpreted (and implemented) as directly or indirectly processing an MST computed in the space of *mutual reachability distances w.r.t.* a smoothing parameter m_{pts}. This MST implicitly represents a density-based hierarchical organization of the data, which can be influenced by the value of m_{pts} used to estimate the density at the data points. For this reason, users typically need to explore multiple parameter settings to gain a deeper understanding of the data and select suitable values of m_{pts} for a given application. Since the framework proposed in [18] decoupled the construction of this type of density-based MST from the actual algorithms, the work presented here can be applied to all the algorithms based on that framework.

A density-based hierarchical organization (*w.r.t.* m_{pts}) of a data set $\mathbf{X}$ with n objects is based on the following concepts:

- **The core-distance $\mathbf{c_{m_{pts}}}(\cdot)$.** The core distance $c_{m_{pts}}(p)$, for each point $p \in \mathbf{X}$, is the smallest radius ε around p that covers m_{pts} other points, *i.e.*, given an underlying distance function $d(\cdot,\cdot)$, it is the distance between p and its

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

$mpts$-nearest neighbor[1]: $c_{m_{pts}}(p) = d(p, m_{pts}\text{-}NN(p))$. The core-distance of a point p can be seen as the inverse of an unnormalized density estimate at the point p [11].

- **The mutual reachability distance $\mathbf{mrd_{m_{pts}}}(\cdot,\cdot)$.** The mutual reachability distance, $mrd_{m_{pts}}(p,q)$, between a pair of data points p, q (*w.r.t.* m_{pts}) is defined as:

$$mrd_{m_{pts}}(p,q) = max\{c_{m_{pts}}(p), c_{m_{pts}}(q), d(p,q)\} \quad (1)$$

Notice that $mrd_{m_{pts}}(p,q)$ is the smallest distance at which both points, p and q, are in each other's ε-neighborhood, and both ε-neighborhoods contain at least m_{pts} points. This distance can be seen as the inverse of an unnormalized density (estimate), representing the maximum density α (w.r.t. a density h) at which both points p and q, and any position along the direct path between p and q, have guaranteed a density estimate corresponding to α in the given data set (based only on the points p, q, and their $mpts$-nearest neighbors).

Conceptually, the process of obtaining a density-based hierarchy *w.r.t.* m_{pts} consists of taking the complete, unweighted graph G of the data set, incorporating mutual reachability distances ($mrd_{m_{pts}}$) as edge weights to obtain a weighted graph $G_{m_{pts}}$, and then computing the MST of this graph. To find the mutual reachability distances, the core distance values for all $p \in \mathbf{X}$ must be computed, which implies running a *k-Nearest-Neighbor* query for each point p, with $k = m_{pts}$ to obtain the k-nearest neighbors of p. Thus, computing the k-nearest neighbors is part of the process of finding the minimum spanning tree in the space of reachability distances.

When a collection of $MSTs$ (encoding multiple density-based hierarchies) for a set of m_{pts} values is required, repeating this process for each value of m_{pts} in this set will require to recompute the $O(n^2)$ edge weights of $G_{m_{pts}}$ by first running a *k-Nearest-Neighbor* (k-NN) query for each point in the data set with k equal to the current m_{pts} value (in order to determine all core distances) and, then, computing the reachability distances for all pairs of points. To speed up the computation of multiple density-based $MSTs$, one can, as a first measure, avoid re-running the *k-Nearest-Neighbor* query for every value of m_{pts} by precomputing and reusing the k_{max}-NN queries, since the k-NN information *w.r.t.* all $k \leq k_{max}$ is contained within the k_{max}-NN.

Using the k_{max}-NN queries to obtain all needed core-distances, however, does not address the high computational cost of computing a possibly large number of $MSTs$ over complete graphs $G_{m_{pts}}$ for the different values of m_{pts}. The computational cost of building an MST increases with the number of edges in the graph it is extracted from. As $G_{m_{pts}}$ has $\Theta(n^2)$ edges, replacing it with a smaller graph would speed up the MST calculations and, consequently, reduce the overall computational cost of obtaining multiple density-based hierarchies. This was first done in [19] by replacing the $G_{m_{pts}}$ with a special relative neighborhood graph (RNG), which was constructed to contain all edges needed to compute the hierarchies for every value of $m_{pts} \in [1, k_{max}]$. Computing an RNG imposes a computational overhead that only pays off if the number of edges is reduced significantly compared to the complete graph and the number of additional hierarchies is large enough. While this may be the case in many application scenarios, this approach has limitations and shortcomings that will be addressed in Section IV-D, including RNG's upper bound on the number of edges, which is $O(n^2)$.

In this work, we show that there is a better graph that can replace $G_{m_{pts}}$ to obtain the hierarchies for every value of $m_{pts} \in [1, k_{max}]$. This graph has $O(n)$ edges assuming $k_{max} << n$, which is a realistic assumption in practice, and its construction requires the dynamic computation of a single density-based MST *w.r.t.* $m_{pts} = k_{max}$, in addition to a k-nearest neighbors graph from the k-NN queries for $k = k_{max}$.

From a computational geometry point of view, the k-NN queries can be used to build an augmented k-nearest neighbors graph $k\text{-}NNG = (E_k, V_k)$ with $V_k = \{p | p \in \mathbf{X}\}$ and $E_k = \{(p,q) | p \in \mathbf{X} \wedge q = i\text{-}NN(p) \, \forall i \in [1,k]\}$. When k-$NN(p)$ is not unique, the graph is augmented with all points tied as the kth neighbor. k-NNGs have the following important properties, which can be used to speed up the computation of multiple density-based $MSTs$:

- Given two k-$NNGs$, i-NNG and j-NNG with $i < j$, it holds true that $i\text{-}NNG \subseteq j\text{-}NNG$. Indeed, j-NNG can be obtained by augmenting i-NNG with edges between every point p and its $(i+1)$th to jth neighbors.
- For a given value of $k = m_{pts}$, all edges of the k-NNG have weights determined by core distances in the complete graph $G_{m_{pts}}$ (Lemma 1), and all edges in $G_{m_{pts}}$ that have weights determined by core distances are also contained in the k-NNG (Lemma 2).

Lemma 1. $(p,q) \in k\text{-}NNG \Rightarrow mrd_k(p,q) = \max\{c_k(p), c_k(q)\}$

Proof:

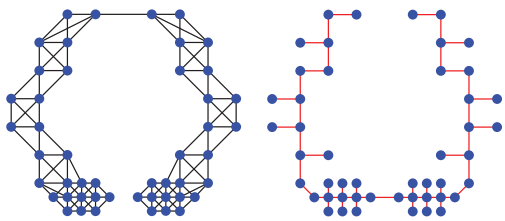
Consider an edge (p,q) in the k-NNG. Based on the definition of the k-NNG, at least one of the points p and q must be among the other point's k nearest neighbors. Therefore, we know that $d(p,q) \leq c_k(p)$ or $d(p,q) \leq c_k(q)$, or both. Given the definition of mutual reachability distance in Equation 1, since the value $d(p,q)$ is smaller than or at most equal to at least one of the *core distances* of p and q, it follows that $mrd_k(p,q) = \max\{c_k(p), c_k(q)\}$. ■

Lemma 2. $\forall m_{pts} \leq k$: $mrd_{m_{pts}}(p,q) = \max\{c_{m_{pts}}(p), c_{m_{pts}}(q)\} \Rightarrow (p,q) \in k\text{-}NNG$

Proof: Let us assume that $mrd_{m_{pts}}(p,q) = max\{c_{m_{pts}}(p), c_{m_{pts}}(q)\}$. Given the definition of the mutual reachability distance in Equation 1, it follows that $c_{m_{pts}}(p) \geq d(p,q)$ or $c_{m_{pts}}(q) \geq d(p,q)$, or both. Hence, at least one of these points must be among the m_{pts}-nearest neighbors of the other, *i.e.*, $q = i\text{-}NN(p)$ and/or $p = i\text{-}NN(q)$ for some $i \in [1, m_{pts}]$, which implies $(p,q) \in m_{pts}\text{-}NNG$.

[1]The m_{pts}-$NN(p)$ may not be unique. However, the core-distance is not affected by ties in m_{pts}-$NN(p)$; all tied points have the same distance to p.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.



(a) k-NNG, k=4 (b) $MST_{m_{pts}}$, m_{pts}=4

Fig. 1: Example of a k-NNG in (a) that does not contain the MST in (b) for a given $G_{m_{pts}}$, where $k = m_{pts} = 4$.

Since $m_{pts} \leq k$, it follows that m_{pts}-$NNG \subseteq k$-NNG and, accordingly, $(p, q) \in k$-NNG. ■

Lemma 2 implies that, for any value of $m_{pts} \in [1, k_{max}]$, all edges that are in an MST of $G_{m_{pts}}$ (referred to as $MST_{m_{pts}}$) and whose weights are determined by a core-distance can be obtained from k_{max}-NNG rather than from the complete graph $G_{m_{pts}}$. All other edges of such an $MST_{m_{pts}}$ (which are not determined by a core-distance), in turn, are not guaranteed to be contained in k_{max}-NNG. In general, for a given $k = m_{pts}$, the k-NNG does not contain an MST of $G_{m_{pts}}$, even if k-NNG is a connected graph — which is why all density-based methods, in addition to computing k-nearest neighbors, must also explicitly compute an MST over $G_{m_{pts}}$. As an example, Figure 1 illustrates that a connected k-NNG by itself may not suffice to find $MSTs$ of $G_{m_{pts}}$.

All the edges of an $MST_{m_{pts}}$ of $G_{m_{pts}}$ that are not guaranteed to be contained in k_{max}-NNG (i.e., those that are not determined by a core-distance) can be shown to be necessarily contained in $MST_{k_{max}}$ though, where $MST_{k_{max}}$ is an MST of $G_{k_{max}}$ ($G_{m_{pts}}$ with $m_{pts} = k_{max}$). Alongside with Lemma 2, this means that $MST_{m_{pts}}$ must be contained in the union of k_{max}-NNG and $MST_{k_{max}}$ for any value of $m_{pts} \in [1, k_{max}]$, as formalized in Theorem 1.

Theorem 1. *Let M_i be the set of all possible $MSTs$ of G_i, and let MST_k be any MST of G_k, then $\forall i < k : \exists MST_i \in M_i \ : \ MST_i \subseteq MST_k \cup k$-$NNG$.*

Proof: Let us consider an edge (p, q) in an $MST_i^* \in M_i$ that connects two sets of points P and Q. The weight of the edge (p, q) is defined by $mrd_{m_{pts}}(p, q)$, which is the maximum of the underlying distance between p and q and their core distances, as defined in Equation 1. Due to the cut property of $MSTs$, (p, q) must have the smallest weight among all edges connecting sets P and Q in G_i. There are two scenarios:

(1) $(p, q) \in MST_k$
(2) $(p, q) \notin MST_k$

In scenario (1), since $(p, q) \in MST_k$, it follows trivially that $(p, q) \in MST_k \cup k$-NNG. In scenario (2), MST_k must have a different edge (p', q') that connects sets P and Q while having the smallest weight among edges that connect these sets (note that $MSTs$ are connected graphs). Consequently, the following two statements are true (i) $mrd_i(p, q) \leq mrd_i(p', q')$ (otherwise (p, q) could not be the edge with the smallest weight connecting P and Q in MST_i^*) and (ii) $mrd_k(p, q) \geq mrd_k(p', q')$ (otherwise (p', q') could not be the edge with the smallest weight connecting P and Q in MST_k). Now, notice that the weight of an edge cannot decrease when the value of m_{pts} increases. When increasing the value of m_{pts}, core distances $c_{m_{pts}}(\cdot)$ can only increase and the base distances between points $d(\cdot, \cdot)$ do not change. Consequently, mutual reachability distances (edge weights), defined in Equation 1, can only remain the same or increase. Hence, when m_{pts} increases from i to k ($i < k$ as per assumption in the theorem) one of the following must hold true:

(a) $mrd_i(p, q) < mrd_k(p, q)$
(b) $mrd_i(p, q) = mrd_k(p, q)$

If (a) is true, as the underlying (base) distance between two points does not depend on m_{pts}, the mutual reachability distance $mrd_k(p, q)$ must be determined by the core distance of either p or q, *i.e.*, $mrd_k(p, q) = max\{c_k(p), c_k(q)\}$. From Lemma 2 with $m_{pts} = k$ it follows then that $(p, q) \in k$-NNG and, thus, $(p, q) \in MST_k \cup k$-NNG. If (b) holds true instead, then from (i) and (ii) above we have $mrd_k(p', q') \leq mrd_k(p, q) = mrd_i(p, q) \leq mrd_i(p', q')$, but since $i < k$ and $mrd_{m_{pts}}$ cannot decrease when m_{pts} increases, it follows that $mrd_k(p', q') = mrd_i(p, q) = mrd_i(p', q')$. In this case, replacing (p, q) in MST_i^* with (p', q') will result in another $MST_i' \in M_i$, because the total weight of MST_i^* and MST_i' are the same. MST_i^* includes the edge (p', q') and by assumption $(p', q') \in MST_k$, and hence $(p', q') \in MST_k \cup k$-$NNG$.

It follows that, given an MST of G_k, MST_k, and an MST of G_i, $MST_i^* \in M_i$, with $k > i$, we can construct a weighted graph MST_i' in the following way: (1) include all edges and edge weights of edges $(p, q) \in MST_i$ that belong to $MST_k \cup k$-NNG (as per scenarios (1) or (2)-a); (2) replace all edges $(p, q) \in MST_i$ that do not belong to $MST_k \cup k$-NNG with edges (p', q') that must exist in $MST_k \cup k$-NNG (as per scenario (2)-b), connecting the same two subsets of points as (p, q) and having the same edge weight as (p, q). This graph MST_i' is an MST of G_i, *i.e.*, $MST_i' \in M_i$ and $MST_i' \subseteq MST_k \cup k$-$NNG$. Thus there exists an $MST_i \in M_i$ such that $MST_i \subseteq MST_k \cup k$-$NNG$. ■

Theorem 1 supports that the complete graph $G_{m_{pts}}$ can be replaced by $MST_k \cup k$-NNG when computing additional $MST_{m_{pts}}$ results *w.r.t.* $m_{pts} < k$, with potential for major runtime and memory savings. In Theorem 2, we show that $MST_k \cup k$-NNG is also a valid replacement for the Relative Neighborhood Graph (RNG), currently used by a state-of-the-art method (see Section II).

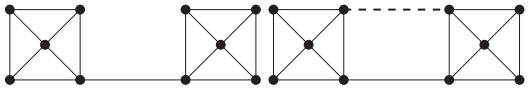
Theorem 2. $MST_k \cup k$-$NNG \subseteq RNG_k$.

Proof: For any edge $(p, q) \in MST_k \cup k$-NNG, we must consider two (not mutually exclusive) scenarios:

(1) $(p, q) \in MST_k$
(2) $(p, q) \in k$-NNG

For scenario (1), it has been proven by Neto *et al* [19] that the MST_k is a sub-graph of RNG_k and, consequently, every

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.



(a) $MST_k \cup k\text{-}NNG$, $k = 3$ (b) RNG_k, $k = 3$

Fig. 2: Example for Theorem 2

edge in the MST_k must also be in the RNG_k.

For scenario (2), we need to prove that $k\text{-}NNG \subseteq RNG_k$. According to Lemma 1, if two points p and q are adjacent in the $k\text{-}NNG$, then the mutual reachability distance between them *w.r.t.* $m_{pts} = k$ is determined by the larger of their core distances. Let us assume, without lost of generality, that $c_k(p) \geq c_k(q)$, which yields $mrd_k(p,q) = c_k(p)$. Due to the properties of the max function, it trivially holds that $c_k(p) \leq \max\{c_k(p), c_k(t), d(p,t)\}\, \forall t$, which can be rewritten as:

$$c_k(p) \leq mrd_k(p,t)\, \forall t \tag{2}$$

Again because of the properties of the max function, we are able to replace the right-hand side of Inequality 2 by a $\max$ function that takes as argument the term $mrd_k(p,t)$ and any other term, including $mrd_k(q,t)$, *i.e.*:

$$c_k(p) \leq \max\{mrd_k(p,t), mrd_k(q,t)\}\, \forall t \tag{3}$$

As per our assumption that $mrd_k(p,q) = c_k(p)$, Inequality 3 can be rewritten only in terms of *mutual reachability distances*:

$$mrd_k(p,q) \leq \max\{mrd_k(p,t), mrd_k(q,t)\}\, \forall t \tag{4}$$

Note that Inequality 4 corresponds to the formulation of the RNG_k and determines that points p and q are *relative neighbors* in the space of mutual reachability distances [19], [20]. This is also true assuming $mrd_k(p,q) = c_k(q)$ instead (by just swapping p and q). This result means that the edge (p,q) assumed to be in the $k\text{-}NNG$ must necessarily be in the RNG_k, *i.e.*, $k\text{-}NNG \subseteq RNG_k$.

Combined, the results for cases (1) and (2) imply that the union of $k\text{-}NNG$ and MST_k is a sub-graph of RNG_k. ■

In order to show that, in general, the opposite is not true, i.e., $RNG_k \nsubseteq MST_k \cup k\text{-}NNG$, we present a counter-example in Figure 2, where the RNG_k (Figure 2b) has one more edge than the $k\text{-}NNG \cup MST_k$ (indicated by a dashed line).

IV. Proposed Approach

We will refer to the graph $MST_k \cup k\text{-}NNG$ as the *CORE-distance based Spanning Graph*, $CORE\text{-}SG_k$. The $CORE\text{-}SG_{m_{pts}}$ is an important replacement graph for the complete graph $G_{m_{pts}}$ when computing density-based $MSTs$ and their encoded density-based data hierarchies for multiple m_{pts} values, because its number of edges is $\Theta(nm_{pts})$, in contrast to $\Theta(n^2)$ for the $G_{m_{pts}}$ or $O(n^2)$ for the RNG, and its construction only requires information readily available from a one-off dynamic pre-computation of a single density-based MST *w.r.t.* the largest value of m_{pts} in a desired range.

A. Algorithm

Based on Theorem 1, we present in Algorithm 1 an approach to efficiently support the task of computing multiple density-based minimum spanning trees $MST_{m_{pts}}$ for different values of m_{pts}, up to a maximum value k_{max}. In part (A), both $MST_{k_{max}}$ and, without much additional effort, $CORE\text{-}SG_{k_{max}} = MST_{k_{max}} \cup k_{max}\text{-}NNG$ are constructed. Any MST algorithm can be applied to build an $MST_{k_{max}}$ over the complete, "virtual"[2] graph $G_{k_{max}}$, whose edge weights are mutual reachability distances *w.r.t.* $m_{pts} = k_{max}$. To compute these mutual reachability distances, the core-distances for each point have to be determined and, to determine these core-distances, the k_{max}-nearest neighbors of each point have to be found. $MST_{k_{max}}$ represents one part of $CORE\text{-}SG_{k_{max}}$. Computing $k_{max}\text{-}NNG$ can be done "on-the-fly" during the mutual reachability calculation by starting with an empty graph and then adding to it all edges of $G_{k_{max}}$ whose weights are determined by core distances *w.r.t.* $m_{pts} = k_{max}$. Another option is to first perform k_{max}-nearest neighbor queries to find for each point the set of points within its k_{max}-nearest neighbor distance and to store the results so that they can be used for both the $MST_{k_{max}}$ computation (to determine core- and mutual reachability distances) as well as for the construction of $k_{max}\text{-}NNG$. Note that we only need $CORE\text{-}SG$ as an undirected graph since the edge weight of any edge between two points is determined by their mutual reachability distance, which does not depend on the "direction" of an edge. Therefore, we already build the $k\text{-}NNG$ as an undirected graph, which leads to a slight reduction in the total number of edges as compared to a directed k-nearest neighbor graph, since there is only one edge (instead of two edges) for every pair of mutual k-nearest neighbors.

Algorithm 1 Computation of Multiple Density-Based $MSTs$

Require: X: data set; $< k_1, \ldots, k_{max} >$: increasing sequence of m_{pts} values;

(A) Simultaneously construct $\mathbf{MST_{k_{max}}}$ and $\mathbf{CORE\text{-}SG_{k_{max}}}$

1) Find and store the k_{max} nearest neighbors for all data objects in X, and store core distances for each object for $m_{pts} \in [1, \ldots, k_{max}]$.
2) Compute an $MST_{k_{max}}$ for the mutual reachability graph, $G_{k_{max}}$.
3) Compute the undirected $k_{max}\text{-}NNG$ graph, reusing the nearest neighbor queries from Step 1.
4) Build $CORE\text{-}SG_{k_{max}}$ as $k_{max}\text{-}NNG \cup MST_{k_{max}}$.

(B) Obtain the remaining MSTs

1) For m_{pts} in $\{k_1, \ldots, k_{max-1}\}$:
   a) Obtain $CORE\text{-}SG_{m_{pts}}$ by assigning mutual reachability distances *w.r.t.* m_{pts} as edge weights in $CORE\text{-}SG_{k_{max}}$.
   b) Find the $MST_{m_{pts}}$ from $CORE\text{-}SG_{m_{pts}}$.

For part (B) of Algorithm 1, namely, the computation of the additional $MSTs$ for values of m_{pts} smaller than k_{max}, all requested $MSTs$ are computed independently and

[2]Every pair of data points (vertices) are adjacent in this graph, which does not need to be materialized as its edge weights can be dynamically computed.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

directly from a weighted version of $CORE\text{-}SG_{k_{max}}$. For each requested value of $m_{pts} = i < k_{max}$, which can be requested in any order, the edges in the $CORE\text{-}SG_{k_{max}}$ are assigned weights corresponding to the mutual reachability distance *w.r.t.* i, and the MST_i is then extracted from the resulting weighted graph. This independent, unordered computation is easy to implement and can be advantageous for parallelization.

B. Optimization

Although $CORE\text{-}SG_{k_{max}}$ is a very compact replacement for $G_{(\cdot)}$, it is not optimal in that it contains edges that are not part of any of the $MSTs$ *w.r.t.* $m_{pts} < k_{max}$. To find the smallest (i.e. optimal) replacement graph, an optional optimization procedure based on Theorem 1 can be used to select and remove such unnecessary edges from $CORE\text{-}SG_{k_{max}}$. The idea consists in constructing smaller versions (*w.r.t.* the number of edges) of a $CORE\text{-}SG_{m_{pts}}$ in each iteration, for decreasing values of m_{pts}, using *only* the necessary amount of nearest neighbors for the current value of m_{pts}, discarding the edges not found to be part of the respective MST. The result, hereafter called $CORE\text{-}SG^*$, is a graph that is optimal (minimal) as the union of all $MSTs$ of the desired sequence of m_{pts}. This procedure is presented in Algorithm 2.

Algorithm 2 Optional optimization procedure

Require: $CORE\text{-}SG_{k_{max}}$; $MST_{k_{max}}$; $< k_1, \ldots, k_{max} >$: increasing sequence of m_{pts} values;

1) $prev_CORE\text{-}SG = CORE\text{-}SG_{k_{max}}$.
2) $CORE\text{-}SG^* = MST_{k_{max}}$.
3) For each m_{pts} in $< k_{max-1}, \ldots, k_1 >$:
   a) Assign mutual reachability distances *w.r.t.* m_{pts} as edge weights in $prev_CORE\text{-}SG$.
   b) Find $MST_{m_{pts}}$ from the weighted $prev_CORE\text{-}SG$, choosing edges that are in $CORE\text{-}SG^*$ in case of ties.
   c) Compute the undirected $m_{pts}\text{-}NNG$ graph, reusing part of the pre-computed nearest neighbor queries.
   d) $prev_CORE\text{-}SG = m_{pts}\text{-}NNG \cup MST_{m_{pts}}$.
   e) $CORE\text{-}SG^* = CORE\text{-}SG^* \cup MST_{m_{pts}}$.

The optimization is optional and may be applied after part (A) of Algorithm 1, either before or as a replacement for part (B). If all the $MSTs$ computed internally to Algorithm 2 are stored, then they can be readily retrieved for any desired value of m_{pts}, so part (B) of Algorithm 1 becomes unnecessary. Alternatively, we can keep only the final $CORE\text{-}SG^*$ from Algorithm 2, which minimizes storage requirements, and then extract the $MSTs$ in part (B) of Algorithm 1 from this graph. Extracting the $MSTs$ from $CORE\text{-}SG^*$ is faster than their extraction from $CORE\text{-}SG_{m_{pts}}$, especially if edges are labeled with the m_{pts} value(s) for which they are part of an $MST_{m_{pts}}$. This can be advantageous for applications that require on-demand, fast retrieval from a small pre-processed graph, but it comes at the price of the additional pre-computation of $CORE\text{-}SG^*$ in Algorithm 2, and the disadvantage that this approach is not as easily parallelizable.

C. Complexity Analysis

Our method starts by computing $MST_{k_{max}}$ and $k_{max}\text{-}NNG$ in order to build $CORE\text{-}SG_{k_{max}}$ with $\Theta(nk_{max})$ edges. These computations are equivalent to a single execution of HDBSCAN* for $m_{pts} = k_{max}$, an algorithm that is analyzed in detail in [11]. Its worst case computational complexity, assuming $m_{pts} << n$ (as expected in practice), is $O(n^2)$ when the pairwise distances are provided, or $O(an^2)$ when the underlying distance function $d(\cdot,\cdot)$ can be computed in $O(a)$ time for each pair of objects. Note that methods such as [23] and [25], which compute an approximate HDBSCAN* result, possibly in a computationally more efficient way, can also be used to build a $CORE\text{-}SG$. However, using an approximate initial MST will lead to additional $MSTs$ that are also only approximate.

After $CORE\text{-}SG_{k_{max}}$ is obtained, edge weight updates are required for each m_{pts} value, with a computational cost of $O(nm_{pts})$. Then, computing the additional $MSTs$ (or checking and retrieving which subsets of edges from $CORE\text{-}SG^*$ are part of each of them) can be done in $O(|E| \log |V|)$ using any known efficient algorithm, e.g., Kruskal's or Prim's algorithm. Since $|E|$ is $O(nm_{pts})$ and $|V| = n$, the computational complexity for extracting one $MST_{m_{pts}}$ is $O(nm_{pts} \log n)$. Considering a range $[k_1, \ldots, k_{max}]$ of m_{pts} values with at most k_{max} values in total will then result in an overall complexity of $O(nk_{max}^2 \log n)$ to calculate all $MSTs$ within this range. With $k_{max}^2 << n$, as expected in real scenarios, our approach is upper bounded by the initial calculation of $CORE\text{-}SG_{k_{max}}$, which is $O(n^2)$ (see above).

D. Comparison with the RNG approach

A graph previously used to replace the $G_{m_{pts}}$ for calculating multiple hierarchies of HDBSCAN* was the RNG [19], [20], which showed significant computational savings in many scenarios. However, the RNG has the following shortcomings compared to the approach proposed in the current paper:

- *Additional computational effort*: the RNG is not an intrinsic part of density-based methods and must be computed independently, which adds a prohibitive asymptotic computational effort of $O(n^3)$. In an attempt to reduce this effort, the well-separated pairs heuristic proposed in [36] can be applied under the assumption that points are in general position, which may result in computational savings, but is still bounded by $O(n^3)$ in the worst case.
- *Geometric constrains*: while the adoption of the well-separated pairs heuristic can often result in computational savings when it is applicable, its assumption imposes additional constraints on the application scenarios: the underlying dissimilarity/distance must be a metric, and it requires direct access to the spatial coordinates of an object, i.e., it cannot be applied to more general data.
- *Size of the graph*: the number of edges in the RNG is usually smaller than in the complete graph $G_{m_{pts}}$, but it is still upper bounded by $O(n^2)$.
- *Inefficiency for high dimensional data*: as the number of dimensions increases, the number of relative neighbors

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

Variables	Values
k_{max}	20, 30, 50
#points	1k, 10k, 50k
#dimensions	4, 8, 16, 32, 64, 128
#clusters	10, 30, 50

TABLE I: Experimental Setup - Base Graph Analysis

(and edges) grows, which makes the RNG less effective [19]. In addition, the effort to construct the RNG approaches the worst case with increasing dimensionality.

In contrast, the $CORE\text{-}SG$ is a much simpler graph that can be obtained straightforwardly from a single run of a density-based algorithm, such as HDBSCAN*. This graph is not restricted to proper distance metrics and has only $\Theta(kn)$ edges, *i.e.*, $\Theta(n)$ if $k << n$, irrespective of the data dimensionality. Additionally, $CORE\text{-}SG$'s properties allow its iterative optimization into the minimal replacement graph $CORE\text{-}SG^*$. The above characteristics render $CORE\text{-}SG$ a much better alternative to the complete graph than RNG.

V. Experimental Evaluation

We implemented our $CORE\text{-}SG$ approach without optimization (Algorithm 1) and with optimization ($CORE\text{-}SG^*$, Algorithm 2), and compared both with a partially filtered RNG that achieve an overall best performance among the RNG approaches [19], [20]. For a fair runtime performance comparison, all the implementations used in this work compute the exact $MST_{m_{pts}}$ of $G_{m_{pts}}$. The $CORE\text{-}SG$, $CORE\text{-}SG^*$ and RNG^* based approaches were implemented in Python/Cython and the experiments were performed on a virtual machine with 16GB RAM running Ubuntu 20.04.

Our experimental evaluation is organized in three parts. In the first part, we discuss the behaviour of the $CORE\text{-}SG$, $CORE\text{-}SG^*$ and RNG^* as base graphs and how their size changes *w.r.t.* the number of edges according to different parameters. In the second part, our analysis focus on runtime and scalability, varying each parameter individually. Lastly, in the third part real data sets are considered in order to evaluate the behaviour of our approaches in real-world scenarios.

A. Base Graph Analysis

Table I displays the parameters that we vary when analyzing the number of edges in the base graphs considered in our experimental evaluation. We assess the behavior of each graph *w.r.t.* k_{max} as well as *w.r.t.* the number of points, dimensions, and clusters in the data. To obtain data sets with varying properties as displayed in Table I, we used the generator from [37]. We compute $MST_{m_{pts}}$ for all $m_{pts} = 1, 2, \cdots, k_{max}$, such that k_{max} corresponds to the size of the m_{pts} range.

Figure 3 shows the number of edges for all combinations of dimensionality, number of points, number of clusters, and k_{max}. Overall, we notice that the numbers of edges of the $CORE\text{-}SG$ and $CORE\text{-}SG^*$ are very stable, and the behavior is similar across the different scenarios. This is explained by the formulation of the $CORE\text{-}SG$, largely based on a $k_{max}\text{-}NN$ graph, which only depends on the value of k_{max} and the size of the data set. The $CORE\text{-}SG^*$ stands out for its smaller and stable size in all scenarios, as the optimization method systematically reduces the number of edges using the results from previous iterations. Our baseline competitor, RNG^*, is not built from the result of a previous computation and, therefore, cannot be similarly optimized.

The edges of RNG^* are partially filtered based on a heuristic that relies solely on information about the $k_{max}\text{-}NN$ neighbors of each point for determining whether two points are relative neighbors or not. Smaller values of k_{max} provide less information about each point's neighborhood, and the filtering heuristic loses effectiveness, particularly as dimensionality increases. As a result, the number of edges in an RNG^* *w.r.t.* smaller values of k_{max} becomes significantly larger than the number of edges of a corresponding $CORE\text{-}SG$.

The effect that increased dimensionality has on an RNG^* has to do with "data sparsity", which favors the presence of relative neighbors in the RNG^*. By adding a larger number of (well-defined and separated) clusters in the data, the amount of "empty space" between pairs of points in different clusters tends to increase, increasing the number of relative neighbors. As expected, this effect gets more noticeable in high-dimensional spaces, which accentuates data sparsity overall. More sparsity reduces the effectiveness of the filtering heuristic, resulting in a higher number of edges in the RNG^* for larger numbers of clusters and higher dimensionality most prominently, as mentioned, for smaller values of m_{pts}.

When increasing the number of points, the number of edges generated in the construction of the RNG^* increases, especially in high-dimensional spaces. In this case, the aforementioned ineffectiveness of the RNG^* filtering heuristic for lower values of k_{max} becomes even more noticeable. This behavior is particularly clear in the subplot corresponding to 50k data points and 50 clusters in Figure 3, where the number of edges in the RNG^* with $k_{max} = 10$ becomes larger than the number of edges with $k_{max} = 30$, or even with $k_{max} = 50$ towards the high end of the dimensionality range.

Figure 4 reports the runtime to construct the RNG^*, $CORE\text{-}SG$ and $CORE\text{-}SG^*$, whose sizes are reported in Figure 3. One can notice that the construction of the RNG^* is much more computationally intensive than the construction of the $CORE\text{-}SG$, except in very low-dimensional data sets. The difference in processing times increases dramatically with increasing dimensionality, increasing data size, and increasing k_{max} value. These results support our claim that constructing a $CORE\text{-}SG$ requires considerably less computational effort than constructing a corresponding RNG^* while being more well-behaved in terms of the number of edges. It is also clear that the optional optimization process to build the $CORE\text{-}SG^*$ added little additional computational effort, displaying a total runtime time close to the original $CORE\text{-}SG$.

B. Runtime and Speedup Analysis

We assess the performance and scalability of our strategy *w.r.t.* the same parameters as in the previous section, but now

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

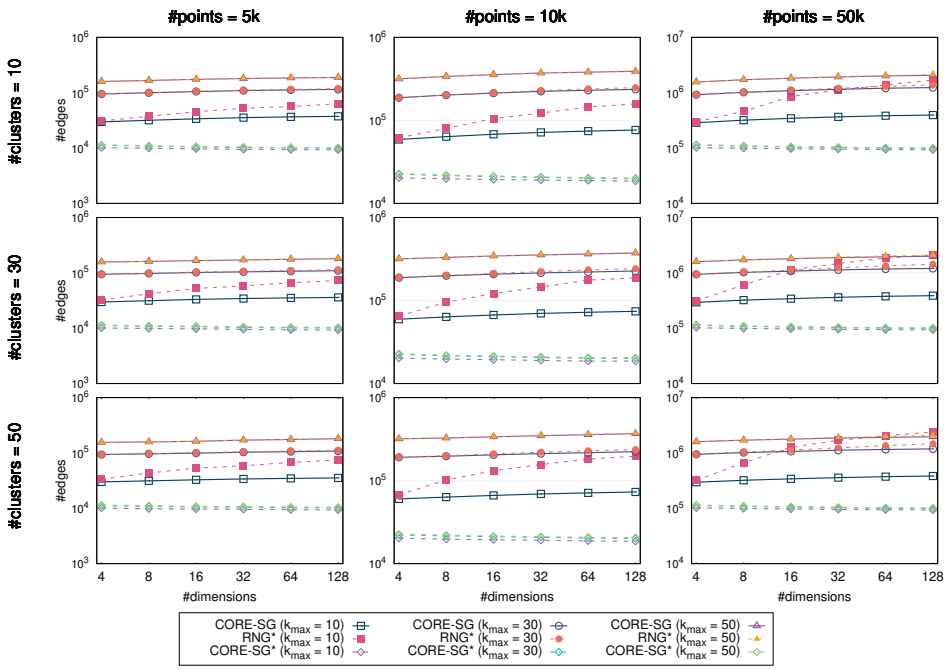


Fig. 3: Number of edges in the RNG^*, $CORE\text{-}SG$, and in the $CORE\text{-}SG^*$.

Variables	Values
k_{max}	20, 40, **60**, 80, 100
#points	1k, 5k, **10k**, 50k, 100k
#dimensions	4, 8, 16, **32**, 64, 128
#clusters	10, **30**, 50

TABLE II: Experimental Setup - Runtime and Speedup

within wider ranges of the parameter values, as displayed in Table II. The entries in bold indicate default values for each parameter, which are kept fixed when assessing the individual effects of varying the values of other parameters.

1) Runtime Analysis: Figure 5 shows the total time (in minutes) to construct the base graphs and k_{max} $MSTs$ for each of the compared methods, varying different parameters.

Varying the number of clusters. Figure V-A shows the runtime of the different approaches when increasing the number of well-separated clusters in a data set of fixed size. Note that $CORE\text{-}SG$ and $CORE\text{-}SG^*$ have similar runtimes, which are much smaller than the runtime of the RNG^* based approach. For instance, for the data set with 50 clusters, our approaches are about 20 times faster. Furthermore, the runtime of the RNG^* based approach increases with the number of clusters, which is likely due to an increasing number of inter-cluster edges in an RNG. Our approaches, on the other hand, are not really affected by the number of clusters.

Varying k_{max}. Figure 5b indicates that the runtime, when increasing k_{max}, grows slowly for each of the compared methods. However, the overall runtime using our $CORE\text{-}SG$ approaches are much lower than using the RNG^*.

Varying the size of the data set. Figure 5c shows the runtimes for data sets of different sizes. As expected, the runtime of all three algorithms increases *w.r.t.* the number of points. While there is no noticeable difference between $CORE\text{-}SG$ and $CORE\text{-}SG^*$, they are substantially faster than RNG^* in all cases. For instance, using $CORE\text{-}SG$ to compute $k_{max} = 60$ $MSTs$ for the data set with $100,000$ points is over 20 times faster than using the RNG^* approach.

Varying the dimensionality of the data set. Figure 5d shows the runtimes for different data dimensionalities. Similar to what happened when increasing the size of the data set, there is no noticeable difference in runtime between $CORE\text{-}SG$ and $CORE\text{-}SG^*$, and the runtimes of all three methods increase with the dimensionality of the data. Once again, while the runtime of the proposed methods increases only slightly, RNG^*'s increases much more substantially and at a faster rate. For instance, using $CORE\text{-}SG$ to compute $k_{max} = 60$ $MSTs$ for the 128-dimensional data set (with $10,000$ points) is over 15 times faster than using the RNG^*.

2) Base graph construction versus construction of additional $MSTs$: The behavior of the total runtime of the different methods presented in Fig. 5 is similar to the observed runtime for construction of the base graphs discussed in Section V-A. This suggests that the total runtime to compute a large number of $MSTs$ using these approaches is dominated by the runtime to construct the underlying base graphs. Here, we show that this is indeed the case, using the $10k$, $50k$, and $100k$ data sets. Table III shows the runtime breakdown into base graph construction and extraction of additional $MSTs$.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

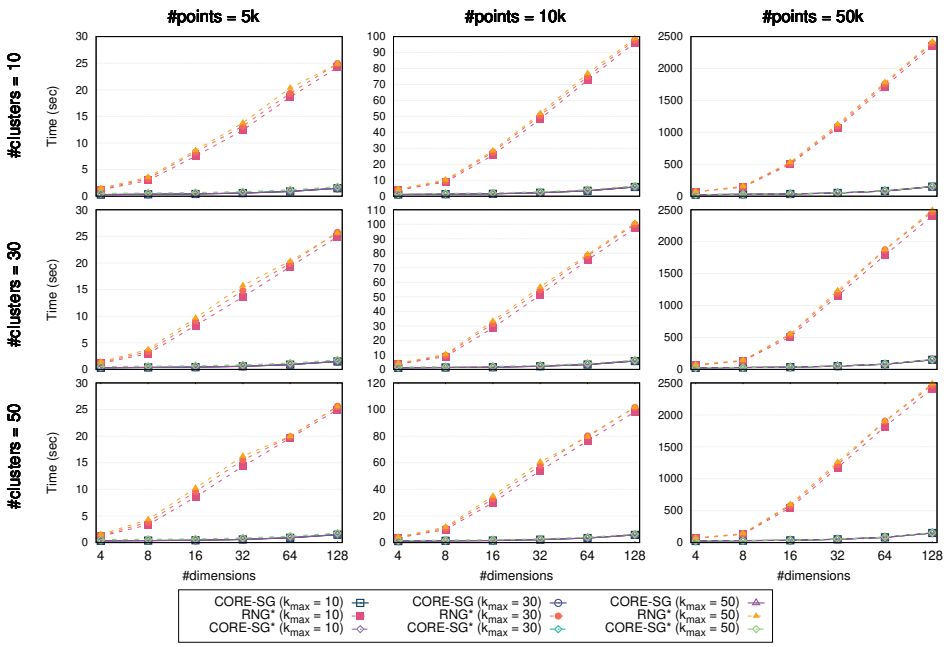
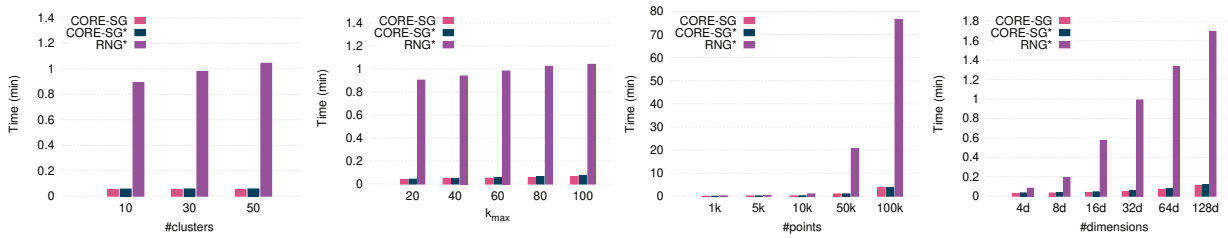


Fig. 4: Runtime to build the RNG^*, $CORE\text{-}SG$, and in the $CORE\text{-}SG^*$.



(a) #clusters vs. Time (b) k_{max} vs. Time (c) #points vs. Time (d) #dimensions vs. Time

Fig. 5: Runtime to build the $CORE\text{-}SG$, $CORE\text{-}SG^*$ and RNG^* and compute k_{max} $MSTs$.

While the runtimes to extract additional $MSTs$ from the base graphs are similar for $CORE\text{-}SG$ and RNG^*, they are nearly halved by using the $CORE\text{-}SG^*$. Note that the runtimes to compute additional $MSTs$ are orders of magnitude smaller than the runtimes to construct the initial base graphs though, thus having less impact on the total time.

3) Speeding-up the Construction of Density-based $MSTs$: In this section, we investigate the speedups that can be accomplished when computing multiple $MSTs$ using the $CORE\text{-}SG$, $CORE\text{-}SG^*$, and RNG^* approaches as compared to the standard approach that uses the complete mutual reachability graph. We measure the ratio between the total runtime to compute k_{max} $MSTs$ from the complete graph to the total runtime to compute the replacement graphs and extract k_{max} $MSTs$ from these graphs. Figure 6 shows the speedups for data sets of different sizes when using different values of k_{max}, which is also the number of computed $MSTs$. The numbers of dimensions and clusters were set to their boldfaced default values in Table II.

For $CORE\text{-}SG$ and $CORE\text{-}SG^*$ there is a noticeable interplay between the data set size and the value of k_{max}. When the ratio of k_{max} over the number of data points is larger, which means that the maximum neighborhood size is relatively large when compared to the data set size, the achieved reduction in graph size (in relation to the complete graph) is not as prominent and, as a result, the speedups are smaller. Conversely, if k_{max} is small compared to the size of the data set (usually the case in practical applications), then the base graphs will be considerably smaller than the complete graph, which translates into larger speedups.

Notice that optimizing $CORE\text{-}SG$ into $CORE\text{-}SG^*$ requires an extra effort and, while the reduction in the number of edges plays an important role in the computation of additional $MSTs$, the overhead of the optional optimization step is still a dominant factor in the total runtime, so $CORE\text{-}SG^*$ produces smaller speedup ratios than $CORE\text{-}SG$, but still noticeably superior to RNG^*. The speedup ratios observed for RNG^* are almost unaffected as k_{max} and the data size increase,

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

Data Set	Base Graph Construction			Construction of k_{max} MSTs			Total Time		
	CORE-SG	CORE-SG*	RNG*	CORE-SG	CORE-SG*	RNG*	CORE-SG	CORE-SG*	RNG*
10k, 32d, 30c, kmax=60	0.0369	0.0506	0.9639	0.0130	0.0068	0.0134	0.0500	0.0574	0.9773
50k, 32d, 30c, kmax=60	0.8640	1.2215	20.4489	0.1037	0.0513	0.1053	0.9677	1.2728	20.5542
100k, 32d, 30c, kmax=60	3.4044	4.7882	76.3006	0.2406	0.1367	0.2509	3.6450	4.9249	76.5515

TABLE III: Runtime (in minutes) for base graph and additional $MSTs$ construction for the proposed and baseline approaches.

but the overhead of constructing this graph dominates to a much larger extent, preventing it from achieving speedup ratios similar to $CORE\text{-}SG$ and $CORE\text{-}SG^*$.

C. Real World Data Sets

We apply our techniques to real-world data sets in different application domains, namely, audio, image, and text. All the pre-processed data sets are made available online and can be downloaded for reproducibility. The FMA data set [38] was created for music analysis and consists of features extracted from a collection of songs from multiple genres and artists. We use two versions of the FMA data set corresponding to different audio features: *chroma* and *MFCC (Mel Frequency Cepstral Coefficient)*. The ImageNet data set [39] has been widely used by the machine learning and computer vision communities. We randomly selected 80 clusters, corresponding to approximately 100k points of this data set, where each point corresponds to a 16×16 color image. Lastly, we apply our method to the *20 Newsgroups* data set [40]. This data set contains the TF-IDF values of terms extracted from Newsgroups articles. For our purpose, we selected the top 1000 terms with the highest TF-IDF values, and applied PCA to reduce the number of dimensions to the 500 principal components that explain 75% of the variance in the data. The size and dimensionality of each data set is shown in Table IV.

Data Set	#points	#dimensions
FMA (Chroma Cens)	106,571	84
FMA (MFCC)	106,574	140
20 Newsgroups	11,314	1000
20 Newsgroups (PCA)	11,314	500
ImageNet	100,610	768

TABLE IV: Real data sets.

Figure 7 shows the total runtime (in minutes) that it takes to construct the base graphs and $k_{max} = 60$ $MSTs$, for each of the considered data sets. As expected, there is no noticeable difference in performance between the $CORE\text{-}SG$ and $CORE\text{-}SG^*$ approaches, and both considerably outperform the RNG^*. This is consistent with the results of our previous experiments involving artificial data sets.

Table V shows the detailed breakdown of the runtime into time for base graph construction and time for construction of additional $MSTs$. Once again, one can notice that the construction of the base graphs dominates the total runtime, which has the $CORE\text{-}SG$ as the fastest approach, closely followed by $CORE\text{-}SG^*$. Like in the experiments with synthetic data, $CORE\text{-}SG^*$ exhibits the best runtimes to extract additional $MSTs$ from the base graphs and these runtimes amount to a small fraction of the time to construct the initial base graph.

So far, we have demonstrated the efficiency of the $CORE\text{-}SG$ based approach as compared to the current state-of-the-art, RNG^*. Next, we will show that (i) the computational effort required to build a $CORE\text{-}SG$ represents only a small overhead over the runtime of a typical density-based method, such as HDBSCAN*, for a single value of m_{pts}, and (ii) producing the result of the same density-based algorithm again for a different value of m_{pts} can be achieved at a tiny fraction of the cost when using a $CORE\text{-}SG$ based approach.

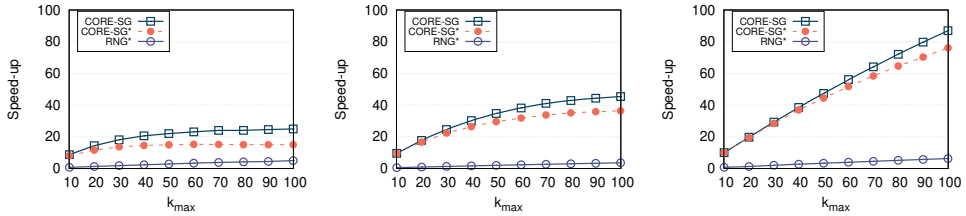
Table VI shows the runtimes to compute a single clustering hierarchy using "plain" HDBSCAN* with $m_{pts} = 60$, as compared to building the $CORE\text{-}SG$ or $CORE\text{-}SG^*$ graphs for $k_{max} = m_{pts} = 60$ and then producing the same HDBSCAN* hierarchy based on the corresponding $MSTs$. Notice that adopting the $CORE\text{-}SG$ adds a minimal overhead associated with its construction, ranging from 0.05% to 0.66%. As discussed earlier, the $CORE\text{-}SG$ construction does not require much more information than what is already computed by density-based algorithms for unsupervised/semi-supervised learning. Building $CORE\text{-}SG^*$ requires an extra effort associated with the optimization process in Algorithm 2, which increased the overhead ratios in Table VI, but without affecting the order of magnitude of the total runtimes.

Table VII shows the runtimes required to compute a second clustering hierarchy, *w.r.t.* a different value of the m_{pts} parameter ($m_{pts} = 30$), for each of the real data sets. When using "plain" HDBSCAN*, the cost is essentially the same as the initial cost required to compute the first hierarchy ($m_{pts} = 60$). In contrast, with both $CORE\text{-}SG$ based approaches, we can compute any other hierarchy in a tiny fraction of the time required to compute the first one, leading e.g. to a speedup factor of about 7700 times for the ImageNet data.

VI. Conclusion

We have proposed a strategy to replace the complete graph representing the density estimates between pairs of points in a data set with a simple and much smaller spanning graph, the $CORE\text{-}SG$, for the purpose of obtaining a collection of density-based $MSTs$ *w.r.t.* multiple values of the parameter m_{pts}, which controls the smoothness of the density estimates. We proved that the $CORE\text{-}SG$ corresponding to a given m_{pts} value contains all the information required to find $MSTs$ for any smaller values of m_{pts} and that the $CORE\text{-}SG$ can never be larger than the relative neighborhood (RNG) graph, the current state-of-the-art solution for this problem. We also devised an optional optimization process to iteratively reduce the $CORE\text{-}SG$ size to the minimal $CORE\text{-}SG*$, which can be a better alternative for applications with strict

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.



(a) #points = 1k (b) #points = 5k (c) #points = 50k

Fig. 6: Speedup relative to the computation of additional $MSTs$ for $CORE\text{-}SG$, $CORE\text{-}SG^*$, and RNG^* based approaches.

Data Set	Base Graph Construction			Construction of k_{max} MSTs			Total Time		
	CORE-SG	CORE-SG*	RNG*	CORE-SG	CORE-SG*	RNG*	CORE-SG	CORE-SG*	RNG*
FMA(C)	7.51	7.96	78.00	0.27	0.15	0.29	7.77	8.11	78.29
FMA(M)	12.07	12.60	83.51	0.25	0.14	0.26	12.32	12.75	83.77
20NG(P)	0.43	0.63	4.08	0.09	0.08	0.09	0.52	0.72	4.17
20NG	0.86	0.96	7.17	0.07	0.11	0.07	0.92	1.07	7.24
IMGN	56.14	56.60	384.25	0.37	0.26	0.38	56.51	56.86	384.63

TABLE V: Runtime breakdown for real datasets (in minutes).

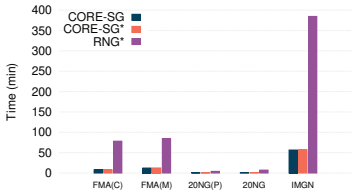


Fig. 7: Runtime to build $CORE\text{-}SG$, $CORE\text{-}SG^*$ and RNG^*, and compute $k_{max} = 60$ $MSTs$ for real data sets.

Data Set	HDBSCAN*	CORE-SG HDBSCAN*	Overhead	CORE-SG* HDBSCAN*	Overhead
FMA(C)	7.36	7.39	0.33%	7.62	3.44%
FMA(M)	12.13	12.16	0.21%	12.39	2.13%
20NG(P)	0.43	0.44	0.66%	0.64	48.26%
20NG	0.85	0.86	0.41%	0.97	13.26%
IMGN	56.48	56.50	0.05%	56.79	0.56%

TABLE VI: Runtime in minutes to compute a first density-based clustering hierarchy with HDBSCAN*, with and without $CORE\text{-}SG$/$CORE\text{-}SG^*$, and the corresponding overheads.

storage constraints and requiring on-demand, fast real-time retrieval of multiple density-based MSTs. Our proposal can be applied to a collection of algorithms that process an MST of the data in the space of density estimates given by mutual reachability distances. We demonstrated experimentally that both optimized and non-optimized versions of our new

Data Set	HDBSCAN*	CORE-SG HDBSCAN*	Speedup	CORE-SG* HDBSCAN*	Speedup
FMA(C)	7.37	0.008	874.57	0.006	1292.03
FMA(M)	12.27	0.009	1358.63	0.006	1972.46
20NG(P)	0.43	0.002	176.89	0.001	293.83
20NG	0.85	0.001	667.38	0.001	852.35
IMGN	56.62	0.011	5002.86	0.007	7786.86

TABLE VII: Runtime in minutes to compute a second density-based clustering hierarchy with HDBSCAN*, with and without $CORE\text{-}SG$/$CORE\text{-}SG^*$, and the corresponding speedups.

proposal significantly outperform the state-of-the-art approach. Overall, the $CORE\text{-}SG$ can be easily adopted by data science practitioners since it is more efficient, more widely applicable, and much easier to implement than the previous RNG based approach. In our experimental evaluation, we have shown that using the $CORE\text{-}SG$ (i) is very stable in terms of the number of edges regarding different factors, (ii) it is much faster to compute than the relative neighbor graph, and (iii) it can speed up the computation of, for instance, additional density-based clustering hierarchies by factors of hundreds to thousands.

In the future we plan to adapt the $CORE\text{-}SG$ approach to distributed and scalable environments, in order to deal with distributed data and data that cannot be processed on a single machine. Another interesting direction for future work is to investigate how a $CORE\text{-}SG$ could be updated dynamically when the data set is not static and may change over time. Orthogonal to the aforementioned lines of investigation, we conjecture that our method could also be combined with spectral clustering to potentially explore multiple solutions for a variety of connected graphs, as very preliminarily discussed in Section II. Also related to this idea, $CORE\text{-}SG$ could be given as input to spectral clustering methods to seek density-based clusters embedded in the graph. Furthermore, in large scale applications where the user is willing to trade some accuracy for additional computational gains, using approximate k-nearest neighbor search combined with the proposed $CORE\text{-}SG$ could be considered. Lastly, the ability to obtain results for a range of parameter values of a density-based method in about the same time it takes to compute a single result with the "plain" method, opens the door to investigate new parameter selection strategies that are based on results.

Acknowledgment

This study was partially funded by NSERC, FAPESP - Grant 2019/09817-6, and Serasa Experian.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] R. Sarkhel and A. Nandi, "Improving information extraction from visually rich documents using visual span representations," *Proc. VLDB Endow.*, vol. 14, no. 5, p. 822–834, Jan. 2021. [Online]. Available: https://doi.org/10.14778/3446095.3446104

[2] J. Ding, V. Nathan, M. Alizadeh, and T. Kraska, "Tsunami: A learned multi-dimensional index for correlated data and skewed workloads," *Proc. VLDB Endow.*, vol. 14, no. 2, p. 74–86, Oct. 2020. [Online]. Available: https://doi.org/10.14778/3425879.3425880

[3] G. Li, X. Zhou, S. Li, and B. Gao, "Qtune: A query-aware database tuning system with deep reinforcement learning," *Proc. VLDB Endow.*, vol. 12, no. 12, p. 2118–2130, Aug. 2019. [Online]. Available: https://doi.org/10.14778/3352063.3352129

[4] Y. Zhang and A. Kumar, "Panorama: A data system for unbounded vocabulary querying over video," *Proc. VLDB Endow.*, vol. 13, no. 4, p. 477–491, Dec. 2019. [Online]. Available: https://doi.org/10.14778/3372716.3372721

[5] D. Wen, L. Qin, Y. Zhang, L. Chang, and X. Lin, "Efficient structural graph clustering: an index-based approach," *The VLDB Journal*, vol. 28, no. 3, pp. 377–399, Jun 2019. [Online]. Available: https://doi.org/10.1007/s00778-019-00541-4

[6] A. C. A. Neto, M. A. Nascimento, J. Sander, and R. J. G. B. Campello, "Mustache: A multiple clustering hierarchies explorer," *Proc. VLDB Endow.*, vol. 11, no. 12, p. 2058–2061, Aug. 2018. [Online]. Available: https://doi.org/10.14778/3229863.3236259

[7] R. J. G. B. Campello, P. Kröger, J. Sander, and A. Zimek, "Density-based clustering," *WIREs Data Mining and Knowledge Discovery*, vol. 10, no. 2, p. e1343, 2020. [Online]. Available: https://doi.org/10.1002/widm.1343

[8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD'96. AAAI Press, 1996, p. 226–231.

[9] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "Optics: Ordering points to identify the clustering structure," in *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '99. New York, NY, USA: ACM, 1999, pp. 49–60. [Online]. Available: http://doi.acm.org/10.1145/304182.304187

[10] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining*, ser. Lecture Notes in Computer Science, J. Pei, V. S. Tseng, L. Cao, H. Motoda, and G. Xu, Eds., vol. 7819. Berlin, Heidelberg: Springer, 2013, pp. 160–172. [Online]. Available: https://doi.org/10.1007/978-3-642-37456-2_14

[11] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection," *ACM Trans. Knowl. Discov. Data*, vol. 10, no. 1, pp. 5:1–5:51, Jul. 2015. [Online]. Available: http://doi.acm.org/10.1145/2733381

[12] W. Savoie, T. A. Berrueta, Z. Jackson, A. Pervan, R. Warkentin, S. Li, T. D. Murphey, K. Wiesenfeld, and D. I. Goldman, "A robot made of robots: Emergent transport and control of a smarticle ensemble," *Science Robotics*, vol. 4, no. 34, 2019. [Online]. Available: https://robotics.sciencemag.org/content/4/34/eaax4316

[13] T. M. Norman, M. A. Horlbeck, J. M. Replogle, A. Y. Ge, A. Xu, M. Jost, L. A. Gilbert, and J. S. Weissman, "Exploring genetic interaction manifolds constructed from rich single-cell phenotypes," *Science*, vol. 365, no. 6455, pp. 786–793, 2019. [Online]. Available: https://science.sciencemag.org/content/365/6455/786

[14] I. Djonlagic, S. Mariani, A. L. Fitzpatrick, V. M. G. T. H. Van Der Klei, D. A. Johnson, A. C. Wood, T. Seeman, H. T. Nguyen, M. J. Prerau, J. A. Luchsinger, J. M. Dzierzewski, S. R. Rapp, G. J. Tranah, K. Yaffe, K. E. Burdick, K. L. Stone, S. Redline, and S. M. Purcell, "Macro and micro sleep architecture and cognitive performance in older adults," *Nature Human Behaviour*, vol. 5, no. 1, pp. 123–145, Jan 2021. [Online]. Available: https://doi.org/10.1038/s41562-020-00964-y

[15] L. A. Miccio and G. A. Schwartz, "Mapping chemical structure–glass transition temperature relationship through artificial intelligence," *Macromolecules*, vol. 54, no. 4, pp. 1811–1817, Feb 2021. [Online]. Available: https://doi.org/10.1021/acs.macromol.0c02594

[16] Logan, C. H. A. and Fotopoulou, S., "Unsupervised star, galaxy, qso classification - application of hdbscan," *A&A*, vol. 633, p. A154, 2020. [Online]. Available: https://doi.org/10.1051/0004-6361/201936648

[17] J. C. Gertrudes, A. Zimek, J. Sander, and R. J. G. B. Campello, "A unified framework of density-based clustering for semi-supervised classification," in *Proceedings of the 30th International Conference on Scientific and Statistical Database Management, SSDBM 2018, Bozen-Bolzano, Italy, July 09-11, 2018*, D. Sacharidis, J. Gamper, and M. H. Böhlen, Eds. ACM, 2018, pp. 11:1–11:12. [Online]. Available: https://doi.org/10.1145/3221269.3223037

[18] ——, "A unified view of density-based methods for semi-supervised clustering and classification," *Data Min. Knowl. Discov.*, vol. 33, no. 6, pp. 1894–1952, 2019. [Online]. Available: https://link.springer.com/article/10.1007/s10618-019-00651-1

[19] A. C. A. Neto, J. Sander, R. J. G. B. Campello, and M. A. Nascimento, "Efficient computation and visualization of multiple density-based clustering hierarchies," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 8, pp. 3075–3089, 2021. [Online]. Available: https://doi.org/10.1109/TKDE.2019.2962412

[20] ——, "Efficient computation of multiple density-based clustering hierarchies," in *2017 IEEE International Conference on Data Mining, ICDM 2017, New Orleans, LA, USA, November 18-21, 2017*, V. Raghavan, S. Aluru, G. Karypis, L. Miele, and X. Wu, Eds. IEEE Computer Society, 2017, pp. 991–996. [Online]. Available: https://doi.org/10.1109/ICDM.2017.127

[21] H. Shiokawa, Y. Fujiwara, and M. Onizuka, "Scan++: Efficient algorithm for finding clusters, hubs and outliers on large-scale graphs," *Proc. VLDB Endow.*, vol. 8, no. 11, p. 1178–1189, Jul. 2015. [Online]. Available: https://doi-org.ez31.periodicos.capes.gov.br/10.14778/2809974.2809980

[22] A. Lulli, M. Dell'Amico, P. Michiardi, and L. Ricci, "Ng-dbscan: Scalable density-based clustering for arbitrary data," *Proc. VLDB Endow.*, vol. 10, no. 3, p. 157–168, Nov. 2016. [Online]. Available: http://www.vldb.org/pvldb/vol10/p157-lulli.pdf

[23] L. McInnes and J. Healy, "Accelerated hierarchical density based clustering," in *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*, 2017, pp. 33–42.

[24] M. de Berg, A. Gunawan, and M. Roeloffzen, "Faster dbscan and hdbscan in low-dimensional euclidean spaces," *International Journal of Computational Geometry & Applications*, vol. 29, no. 01, pp. 21–47, 2019. [Online]. Available: https://doi.org/10.1142/S0218195919400028

[25] Y. Wang, S. Yu, Y. Gu, and J. Shun, "Fast parallel algorithms for euclidean minimum spanning tree and hierarchical spatial clustering," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, G. Li, Z. Li, S. Idreos, and D. Srivastava, Eds. ACM, 2021, pp. 1982–1995. [Online]. Available: https://doi.org/10.1145/3448016.3457296

[26] J. A. d. Santos, T. I. Syed, M. C. Naldi, R. J. G. B. Campello, and J. Sander, "Hierarchical density-based clustering using mapreduce," *IEEE Transactions on Big Data*, vol. 7, no. 1, pp. 102–114, 2021. [Online]. Available: https://doi.org/10.1109/TBDATA.2019.2907624

[27] X. Xu, N. Yuruk, Z. Feng, and T. A. J. Schweiger, "Scan: A structural clustering algorithm for networks," in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '07. New York, NY, USA: ACM, 2007, p. 824–833. [Online]. Available: https://doi.org/10.1145/1281192.1281280

[28] J. M. González-Barrios and A. J. Quiroz, "A clustering procedure based on the comparison between the k nearest neighbors graph and the minimal spanning tree," *Statistics and Probability Letters*, vol. 62, no. 1, pp. 23–34, 2003. [Online]. Available: https://doi.org/10.1016/S0167-7152(02)00421-2

[29] A. S. Arefin, C. Riveros, R. Berretta, and P. Moscato, "knn-mst-agglomerative: A fast and scalable graph-based data clustering approach on gpu," in *2012 7th International Conference on Computer Science Education (ICCSE)*, 2012, pp. 585–590.

[30] A. S. Arefin, C. Riveros, R. Berretta, and P. Moscato, "The mst-knn with paracliques," in *Artificial Life and Computational Intelligence*, S. K. Chalup, A. D. Blair, and M. Randall, Eds. Cham: Springer International Publishing, 2015, pp. 373–386.

[31] A. K. Jain and R. C. Dubes, *Algorithms for Clustering Data*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc., 1988.

[32] H.-P. Kriegel, P. Kröger, J. Sander, and A. Zimek, "Density-based clustering," *WIREs Data Mining and Knowledge Discovery*, vol. 1, no. 3, pp. 231–240, 2011. [Online]. Available: https://doi.org/10.1002/widm.30

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.

[33] S. Hess, W. Duivesteijn, P. Honysz, and K. Morik, "The spectacl of nonconvex clustering: A spectral approach to density-based clustering," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019, pp. 3788–3795. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33013788

[34] U. von Luxburg, "A tutorial on spectral clustering," *Stat. Comput.*, vol. 17, no. 4, pp. 395–416, 2007. [Online]. Available: https://doi.org/10.1007/s11222-007-9033-z

[35] P. Veenstra, C. Cooper, and S. Phelps, "Spectral clustering using the knn-mst similarity graph," in *2016 8th Computer Science and Electronic Engineering Conference, CEEC 2016, Colchester, UK, September 28-30, 2016*. IEEE, 2016, pp. 222–227. [Online]. Available: https://doi.org/10.1109/CEEC.2016.7835917

[36] P. B. Callahan and S. R. Kosaraju, "A decomposition of multidimensional point sets with applications to k-nearest-neighbors and n-body potential fields," *Journal of the ACM*, vol. 42, no. 1, pp. 67–90, 1995.

[37] J. Handl and J. Knowles, "Cluster generators for large high-dimensional data sets with large numbers of clusters," 2005.

[38] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017. [Online]. Available: https://arxiv.org/abs/1612.01840

[39] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*. IEEE Computer Society, 2009, pp. 248–255. [Online]. Available: https://doi.org/10.1109/CVPR.2009.5206848

[40] K. Lang, "Newsweeder: Learning to filter netnews," in *Machine Learning, Proceedings of the Twelfth International Conference on Machine Learning, Tahoe City, California, USA, July 9-12, 1995*, A. Prieditis and S. J. Russell, Eds. Morgan Kaufmann, 1995, pp. 331–339. [Online]. Available: http://people.csail.mit.edu/jrennie/20Newsgroups/

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SAO CARLOS. Downloaded on July 12,2023 at 20:25:48 UTC from IEEE Xplore. Restrictions apply.